

Complete NumPy & Pandas Theory Guide

NumPy: Numerical Python

What is NumPy?

NumPy is the foundational package for numerical computing in Python. It provides a high-performance multidimensional array object and tools for working with these arrays.

Why NumPy exists:

- Python lists are slow for numerical operations (interpreted, dynamically typed)
- NumPy arrays are stored in contiguous memory blocks (C-like performance)
- Vectorized operations eliminate slow Python loops
- Foundation for scientific computing ecosystem (Pandas, SciPy, scikit-learn)

Performance difference:

Python list loop: $O(n)$ operations in Python interpreter
NumPy vectorized: $O(n)$ operations in compiled C code
Speed improvement: 10-100x faster for numerical operations

NumPy Arrays (ndarray)

Core Concept: N-Dimensional Arrays

An ndarray is a grid of values, all of the same type, indexed by a tuple of non-negative integers.

Mathematical representation:

- 0D array (scalar): $a = 5$
- 1D array (vector): $a = [1, 2, 3]$
- 2D array (matrix): $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
- 3D+ array (tensor): Higher dimensional grids

Memory layout:

Array: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

Stored in memory: [1, 2, 3, 4, 5, 6] (row-major, C-style)

When to Use NumPy Arrays vs Python Lists

Use NumPy when:

1. Performing mathematical operations on entire arrays
2. Need memory efficiency (homogeneous data)
3. Require multi-dimensional data structures
4. Need linear algebra operations
5. Working with large datasets (>1000 elements)

Use Python lists when:

1. Data is heterogeneous (mixed types)
 2. Frequently inserting/deleting elements
 3. Small datasets where performance doesn't matter
 4. Need built-in Python methods (append, pop)
-

NumPy Array Creation

Why Different Creation Methods?

`np.zeros()`, `np.ones()`, `np.empty()`

- **When:** Initialize arrays with specific values
- **Why:** Memory pre-allocation is faster than growing arrays
- **Use case:** Creating placeholder arrays for computations

`np.arange()` vs `np.linspace()`

- `np.arange(start, stop, step)`: Specifies step size
 - Use when you know the interval between values
 - Example: `np.arange(0, 10, 2)` → [0, 2, 4, 6, 8]
- `np.linspace(start, stop, num)`: Specifies number of points
 - Use when you need exact number of evenly spaced values
 - Example: `np.linspace(0, 1, 5)` → [0, 0.25, 0.5, 0.75, 1.0]
 - Common in plotting, signal processing

`np.random` functions

- `np.random.rand()`: Uniform distribution [0, 1)

- `np.random.randn()`: Standard normal distribution ($\mu=0$, $\sigma=1$)
 - `np.random.randint()`: Discrete uniform distribution
 - **When:** Testing algorithms, simulations, initializing neural networks
-

Array Indexing and Slicing

Basic Indexing

Mathematical notation:

For 2D array $A[i, j]$:

- i : row index (0 to rows-1)
- j : column index (0 to cols-1)
- $A[i, j]$ accesses element at row i , column j

Why zero-based indexing:

- Memory offset calculation: $\text{address} = \text{base} + (\text{index} \times \text{element_size})$
- Matches pointer arithmetic in C/C++
- Simplifies slice notation

Boolean Indexing (Masking)

Concept: Use boolean arrays to select elements

How it works:

```
python

arr = np.array([1, 2, 3, 4, 5])
mask = arr > 3 # [False, False, False, True, True]
result = arr[mask] # [4, 5]
```

When to use:

- Filtering data based on conditions
- Replacing values conditionally
- Creating subsets for analysis

Mathematical interpretation:

- Boolean mask is an indicator function: $I(x > 3)$
- Returns 1 (True) where condition holds, 0 (False) otherwise

Fancy Indexing

Concept: Use arrays of indices to select elements

```
python

arr = np.array([10, 20, 30, 40, 50])
indices = [0, 2, 4]
result = arr[indices] # [10, 30, 50]
```

When to use:

- Selecting non-contiguous elements
 - Reordering data
 - Gathering results from computed indices
-

Broadcasting

What is Broadcasting?

Broadcasting allows NumPy to perform operations on arrays of different shapes by automatically expanding smaller arrays.

Mathematical concept: Extend smaller array to match larger array's shape without copying data.

Broadcasting rules:

1. If arrays have different numbers of dimensions, pad the smaller array's shape with 1s on the left
2. Arrays are compatible if, for each dimension, sizes are equal or one is 1
3. The dimension with size 1 is stretched to match the other

Example:

```
python

A: (3, 4)  # 3 rows, 4 columns
B: (4,)    # 1D array with 4 elements

Step 1: B becomes (1, 4)
Step 2: B is broadcast to (3, 4) by repeating
Result: A + B works element-wise
```

Visual representation:

```
A = [[1, 2, 3, 4],    B = [10, 20, 30, 40]
     [5, 6, 7, 8],
     [9, 10, 11, 12]]
```

Broadcasting B:

```
[[10, 20, 30, 40],
 [10, 20, 30, 40],
 [10, 20, 30, 40]]
```

Result A + B:

```
[[11, 22, 33, 44],
 [15, 26, 37, 48],
 [19, 30, 41, 52]]
```

Why broadcasting is powerful:

- Avoids explicit loops
- Saves memory (no actual copying)
- Vectorized operations remain fast
- More readable code

Common use cases:

1. Normalizing data: $(X - \text{mean}) / \text{std}$
2. Adding bias in neural networks: $\text{output} + \text{bias}$
3. Distance calculations: $\text{points} - \text{center}$

Universal Functions (ufuncs)

What are ufuncs?

Functions that operate element-wise on arrays, supporting broadcasting and type casting.

Mathematical operations:

- **Arithmetic:** $+$, $-$, $*$, $/$, $**$, $\%$
- **Trigonometric:** $\sin$, $\cos$, $\tan$, $\arcsin$, $\arccos$, $\arctan$
- **Exponential/Log:** $\exp$, $\log$, $\log_{10}$, $\log_2$
- **Comparison:** $>$, $<$, $>=$, $<=$, $==$, $!=$

Why ufuncs are fast:

1. Implemented in compiled C code
2. Operate on entire arrays at once (SIMD instructions)
3. Avoid Python interpreter overhead

Example: Speed comparison

```
python

# Slow: Python loop
result = [math.sqrt(x) for x in arr] # O(n) in Python

# Fast: NumPy ufunc
result = np.sqrt(arr) # O(n) in C, 10-100x faster
```

Aggregation Functions

Statistical Aggregations

Concept: Reduce array to summary statistics

Common aggregations:

1. `sum()`: $\sum x_i$ - Total of all elements
2. `mean()`: $\mu = (\sum x_i)/n$ - Average value
3. `std()`: $\sigma = \sqrt{(\sum (x_i - \mu)^2)/n}$ - Spread of data
4. `var()`: $\sigma^2 = \sum (x_i - \mu)^2/n$ - Variance
5. `min()`/`max()`: Extreme values
6. `median()`: Middle value (50th percentile)
7. `percentile()`: Value below which p% of data falls

Axis parameter explained:

For 2D array (matrix):

- `axis=None` (default): Aggregate entire array
- `axis=0`: Aggregate down columns (result has shape of columns)
- `axis=1`: Aggregate across rows (result has shape of rows)

Visual example:

```
Array: [[1, 2, 3],  
        [4, 5, 6]]
```

```
sum(axis=0): [5, 7, 9] # Column sums: [1+4, 2+5, 3+6]
```

```
sum(axis=1): [6, 15] # Row sums: [1+2+3, 4+5+6]
```

```
sum(): 21 # Total sum
```

Mathematical interpretation:

- `axis=0`: Apply operation along dimension 0 (collapse rows)
- `axis=1`: Apply operation along dimension 1 (collapse columns)

When to use:

- **Exploratory data analysis:** Understand data distribution
 - **Feature engineering:** Create summary features
 - **Validation:** Check for outliers, anomalies
 - **Reporting:** Generate statistics
-

Array Manipulation

Reshaping

Concept: Change array dimensions without changing data

Mathematical constraint:

Original size = New size

If `arr.shape = (m, n)`, then `arr.reshape(p, q)` requires $m \times n = p \times q$

`reshape()` vs `flatten()` vs `ravel()`:

1. `reshape(new_shape)`:
 - Returns view if possible (no copy)
 - Must satisfy size constraint
 - Use when you need specific dimensions
2. `flatten()`:
 - Always returns copy
 - Converts to 1D
 - Safe but slower

3. `ravel()`:

- Returns view if possible
- Converts to 1D
- Faster but changes affect original

When to reshape:

- Preparing data for machine learning (e.g., 28×28 images $\rightarrow$ 784 vector)
- Matrix operations requiring specific dimensions
- Broadcasting compatibility

Transpose

Mathematical concept:

If A is $m \times n$ matrix, then A^T is $n \times m$ matrix
where $(A^T)_{ij} = A_{ji}$

Visual example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

When to use:

- Linear algebra operations ($A^T \times A$)
- Switching row/column orientation
- Dot product calculations: `x.T @ y`

Stacking and Concatenation

`vstack` (vertical stack):

- Stack arrays vertically (row-wise)
- Arrays must have same number of columns
- Use when adding more rows (observations)

`hstack` (horizontal stack):

- Stack arrays horizontally (column-wise)
- Arrays must have same number of rows

- Use when adding more features (variables)

concatenate:

- General purpose concatenation
- Specify axis for direction
- More flexible than vstack/hstack

When to use:

- Combining datasets from multiple sources
 - Adding features or observations
 - Batch processing results
-

Linear Algebra

Matrix Multiplication

Three types of products:

1. **Element-wise (Hadamard product):** $A * B$

$$\begin{aligned} [1, 2] * [5, 6] &= [1 \times 5, 2 \times 6] = [5, 12] \\ [3, 4] \quad [7, 8] \quad [3 \times 7, 4 \times 8] \quad [21, 32] \end{aligned}$$

- Arrays must have same shape
- Use for scaling, masking

2. **Dot product:** $\text{np.dot}(A, B)$ or $A @ B$

$$\text{Mathematical: } (A @ B)_{ij} = \sum_k A_{ik} \times B_{kj}$$

$$\begin{aligned} [1, 2] @ [5, 6] &= [1 \times 5 + 2 \times 7, 1 \times 6 + 2 \times 8] = [19, 22] \\ [3, 4] \quad [7, 8] \quad [3 \times 5 + 4 \times 7, 3 \times 6 + 4 \times 8] \quad [43, 50] \end{aligned}$$

- A is (m, n), B is (n, p) $\rightarrow$ Result is (m, p)
- Inner dimensions must match
- Use for linear transformations, projections

3. **Matrix multiplication:** $\text{np.matmul}(A, B)$

- Similar to dot but handles higher dimensions differently
- Broadcasting for batch operations
- Recommended for modern code

When to use matrix multiplication:

- Linear regression: `X @ weights`
- Neural networks: `activation(W @ input + bias)`
- Coordinate transformations
- Solving systems of equations

Key Linear Algebra Operations

1. Inverse: `np.linalg.inv(A)`

- **Mathematical:** A^{-1} such that $A @ A^{-1} = I$
- **When:** Solving $Ax = b$ as $x = A^{-1}b$
- **Caution:** Only for square, non-singular matrices
- **Numerical stability:** Prefer `solve()` over explicit inverse

2. Determinant: `np.linalg.det(A)`

- **Mathematical:** Scalar value $\det(A)$
- **Interpretation:**
 - $\det(A) = 0 \rightarrow$ Matrix is singular (not invertible)
 - $|\det(A)| =$ volume scaling factor of transformation
- **When:** Checking invertibility, calculating volumes

3. Eigenvalues/Eigenvectors: `np.linalg.eig(A)`

- **Mathematical:** $Av = \lambda v$ where λ is eigenvalue, v is eigenvector
- **Interpretation:** Directions that are only scaled (not rotated) by A
- **When:**
 - Principal Component Analysis (PCA)
 - Stability analysis
 - Google PageRank algorithm
 - Quantum mechanics

4. Solve: `np.linalg.solve(A, b)`

- **Mathematical:** Solve $Ax = b$ for x

- **When:** Systems of linear equations
 - **Why better than inverse:** More numerically stable and faster
-

Pandas: Panel Data

What is Pandas?

Pandas provides data structures for efficiently storing and manipulating labeled, tabular data.

Why Pandas exists:

- NumPy lacks labels (row/column names)
- Need for heterogeneous data (mixed types)
- Database-like operations (group, join, merge)
- Time series functionality
- Missing data handling

Core philosophy:

- **NumPy:** Fast numerical operations on homogeneous arrays
 - **Pandas:** Flexible data manipulation with labels and mixed types
-

Pandas Data Structures

Series: 1D Labeled Array

Mathematical concept:

Series = Vector + Index
Values: [1, 2, 3]
Index: ['a', 'b', 'c']
Result: $a \rightarrow 1, b \rightarrow 2, c \rightarrow 3$

When to use Series:

- Single variable/feature
- Time series data
- Result of aggregations
- Column from DataFrame

Series vs NumPy array:

- Series has index (labels)
- Series allows mixed operations with labels
- Series is built on NumPy (has `.values` attribute)

DataFrame: 2D Labeled Table

Mathematical concept:

DataFrame = Matrix + Row Index + Column Names

	Col1	Col2	Col3
Row1	1	4	7
Row2	2	5	8
Row3	3	6	9

Structure:

- Each column is a Series
- All Series share the same index
- Different columns can have different types

When to use DataFrame:

- Tabular data (like Excel, SQL table)
 - Multiple features per observation
 - Mixed data types
 - Real-world datasets
-

Data Selection: loc vs iloc

The Fundamental Difference

`loc`: Label-based indexing

- Uses row/column labels (names)
- Includes the end point in slices
- Example: `df.loc['row1':'row3']` includes row3

`iloc`: Position-based indexing

- Uses integer positions (0, 1, 2, ...)
- Excludes the end point (Python style)
- Example: `df.iloc[0:3]` includes positions 0, 1, 2

Why two systems?

- Labels are meaningful but can be non-numeric
- Positions are universal but less intuitive
- Need both for flexibility

When to use:

- `loc`: When you know row/column names, time-based slicing
- `iloc`: When position matters, working with unknown labels

Speed comparison:

- `at/iat` for single values: 10x faster than `loc/iloc`
 - Use `at/iat` in loops (though vectorization is better)
-

Handling Missing Data

Types of Missing Data (Statistical Theory)

1. MCAR (Missing Completely at Random):

- Missingness unrelated to any variable
- Safe to drop or impute
- Example: Sensor randomly fails

2. MAR (Missing at Random):

- Missingness related to observed variables
- Can impute using related variables
- Example: Young people don't report income

3. MNAR (Missing Not at Random):

- Missingness related to unobserved value
- Dangerous to impute naively
- Example: High earners don't report income

Strategies for Handling Missing Data

1. Deletion:

```
python

df.dropna()           # Listwise deletion
df.dropna(thresh=0.8*len(df.columns)) # Threshold
```

- **When:** MCAR, small proportion missing (<5%)
- **Pros:** Simple, no bias if MCAR
- **Cons:** Loses information, reduces sample size

2. Imputation:

Mean/Median imputation:

```
python

df.fillna(df.mean())
df.fillna(df.median())
```

- **When:** Numeric data, roughly symmetric distribution
- **Pros:** Preserves sample size
- **Cons:** Underestimates variance, ignores relationships

Forward/Backward fill:

```
python

df.fillna(method='ffill') # Carry last observation forward
df.fillna(method='bfill') # Use next observation
```

- **When:** Time series, ordered data
- **Rationale:** Values change slowly over time
- **Cons:** Not suitable for cross-sectional data

Interpolation:

```
python

df.interpolate(method='linear')
df.interpolate(method='polynomial', order=2)
```

- **When:** Time series, continuous numeric data
- **Mathematical:** Fit function between known points

- **Linear:** $y = y_1 + (y_2 - y_1) \times (x - x_1) / (x_2 - x_1)$

3. Advanced methods:

- Multiple imputation (statistical)
 - KNN imputation (use similar records)
 - Model-based imputation (predict missing values)
-

Data Types and Memory Optimization

Why Data Types Matter

Memory usage example:

int64: 8 bytes per value

int32: 4 bytes per value

int8: 1 byte per value

For 1 million rows:

int64: 8 MB

int32: 4 MB (50% savings)

int8: 1 MB (87.5% savings)

Type selection guide:

Integers:

- `int8`: -128 to 127 (e.g., age, small counts)
- `int16`: -32,768 to 32,767 (e.g., year, medium counts)
- `int32`: ± 2 billion (default choice)
- `int64`: Very large numbers (often overkill)

Floats:

- `float32`: 7 decimal digits precision
- `float64`: 16 decimal digits precision (default)

Strings:

- `object`: Default, flexible but memory-heavy
- `category`: For repeated strings (huge savings)

Category type:

```
python
```

```
df['color'].astype('category')
```

- **When:** Column with repeated values
 - **How it works:** Maps strings to integers internally
 - **Memory saving:** $O(\text{unique_values})$ instead of $O(\text{total_values})$
 - **Example:** 1M rows, 10 unique colors $\rightarrow$ 99% memory reduction
-

GroupBy: Split-Apply-Combine

The GroupBy Paradigm

Conceptual model:

1. **Split:** Divide data into groups based on key(s)
2. **Apply:** Compute function on each group independently
3. **Combine:** Aggregate results back together

Mathematical representation:

For groups $g_1, g_2, \dots, g_k$:

Result = $[f(g_1), f(g_2), \dots, f(g_k)]$

where f could be: sum, mean, count, custom function

Example:

```
python
```



```
df.groupby('category')['sales'].sum()
```

Split:

Category A: [100, 200, 150]

Category B: [300, 250]

Category C: [400, 350, 300]

Apply (sum):

Category A: 450

Category B: 550

Category C: 1050

Combine:

category

A 450

B 550

C 1050

Aggregation vs Transform vs Filter

1. Aggregation:

python

```
df.groupby('group')['value'].mean()
```

- Returns one row per group
- Reduces data size
- Use for summary statistics

2. Transform:

python

```
df.groupby('group')['value'].transform('mean')
```

- Returns same shape as input
- Broadcasts result back to original rows
- Use for group-wise normalization: $(x - \text{group_mean}) / \text{group_std}$

3. Filter:

python

```
df.groupby('group').filter(lambda x: len(x) > 5)
```

- Returns subset of original data
- Keeps entire groups that meet condition
- Use for removing small groups

When to use each:

- **Aggregation:** Creating summary reports, statistics
 - **Transform:** Feature engineering, normalization
 - **Filter:** Data quality checks, outlier removal
-

Merging and Joining

Types of Joins (Set Theory)

Mathematical interpretation using sets:

Given two tables A and B on key column:

- $A = \{a_1, a_2, a_3\}$ (keys in A)
- $B = \{b_1, b_2, b_3\}$ (keys in B)

1. Inner Join: $A \cap B$

- Only matching keys
- Most restrictive
- Use when you need complete information from both sources

2. Left Join: $A \cup (A \cap B) = A$

- All keys from A, matching from B
- Missing B values become NaN
- Use when A is primary dataset

3. Right Join: $(A \cap B) \cup B = B$

- All keys from B, matching from A
- Rarely used (just swap and use left join)

4. Outer Join: $A \cup B$

- All keys from both
- Most inclusive
- Use when you want to preserve all information

Visual example:

```

A:      B:
ID Value_A  ID Value_B
1  100    2  200
2  150    3  250
3  175    4  300

Inner (1∩2): Left (keep A): Right (keep B): Outer (1∪2):
ID Val_A Val_B ID Val_A Val_B ID Val_A Val_B ID Val_A Val_B
2 150  200  1 100  NaN  2 150  200  1 100  NaN
3 175  250  2 150  200  3 175  250  2 150  200
          3 175  250  4 NaN  300  3 175  250
                    4 NaN  300

```

Join Performance Considerations

Complexity:

- Hash join: $O(n + m)$ average case
- Merge join (sorted): $O(n + m)$
- Nested loop: $O(n \times m)$ - avoid!

Best practices:

1. Set indices on join keys if joining repeatedly
2. Use categorical types for join columns with repeated values
3. Sort data before merge join
4. Prefer inner join when possible (smaller result)

Pivot Tables and Reshaping

Pivot Table Theory

Concept: Restructure data from long to wide format with aggregation

Mathematical operation:

Input (long format):

Date	Category	Value
2024-01-01	A	100
2024-01-01	B	200
2024-01-02	A	150
2024-01-02	B	250

Output (wide format after pivot):

	A	B
2024-01-01	100	200
2024-01-02	150	250

When to use:

- Creating cross-tabulations
- Converting from database format to analysis format
- Calculating summary statistics across dimensions
- Preparing data for visualization

Melt (Unpivot): Wide to Long

Inverse operation of pivot

Why melt:

1. **Tidy data principle:** Each variable is a column, each observation is a row
2. **Easier groupby:** When categories are in columns, hard to group
3. **Better for plotting:** Most plotting libraries prefer long format
4. **Database normalization:** Long format is normalized

Example:

Input (wide):

ID	Q1_Sales	Q2_Sales	Q3_Sales
1	100	150	200
2	120	140	180

Output (long after melt):

ID	Quarter	Sales
1	Q1	100
1	Q2	150
1	Q3	200
2	Q1	120
2	Q2	140
2	Q3	180

Time Series Analysis

Why Time Series is Special

Temporal dependencies:

- Today's value depends on yesterday's
- Violates i.i.d. assumption (independent and identically distributed)
- Need special methods: autocorrelation, stationarity

Key Time Series Operations

1. Resampling:

```
python
```

```
df.resample('M').mean() # Upsampling or downsampling
```

Upsampling (increase frequency):

- Daily → Hourly
- Fills missing values with NaN or interpolation
- Use when you need finer granularity

Downsampling (decrease frequency):

- Hourly → Daily, Daily → Monthly
- Requires aggregation function (mean, sum, last)

- Use for summarization, reducing noise

Common frequencies:

- `D`: Daily
- `W`: Weekly
- `M`: Month end
- `MS`: Month start
- `Q`: Quarter end
- `Y`: Year end
- `H`: Hourly
- `T` or `min`: Minute

2. Rolling Windows:

```
python

df.rolling(window=7).mean() # Moving average
```

Mathematical definition:

For window size k :

$$MA(t) = (1/k) \sum_{i=0}^{k-1} x(t-i)$$

Example with $k=3$:

Data: [1, 2, 3, 4, 5]

MA: [-, -, 2, 3, 4]

where 2 = (1+2+3)/3

When to use:

- **Smoothing:** Remove noise from data
- **Trend detection:** See underlying patterns
- **Feature engineering:** Create lagged features
- **Forecasting:** Simple prediction method

Types of rolling statistics:

- `rolling().mean()`: Smooth noise
- `rolling().std()`: Detect volatility changes
- `rolling().sum()`: Cumulative effects
- `rolling().max()`/`min()`: Range over window

3. Shifting:

python

```
df.shift(1)  # Lag by 1 period  
df.shift(-1) # Lead by 1 period
```

When to use:

- Creating lagged features: $df['value_lag1'] = df['value'].shift(1)$
- Calculating returns: $df['return'] = df['price'] / df['price'].shift(1) - 1$
- Comparing periods: $df['change'] = df['value'] - df['value'].shift(1)$

4. Differencing:

python

```
df.diff()  # First difference  
df.diff(2) # Second difference
```

Mathematical:

First difference: $\nabla x(t) = x(t) - x(t-1)$
Second difference: $\nabla^2 x(t) = \nabla x(t) - \nabla x(t-1)$

Purpose:

- Remove trend (make stationary)
- Calculate change/growth rate
- Required for many forecasting models (ARIMA)

5. Percentage Change:

python

```
df.pct_change() #  $(x(t) - x(t-1)) / x(t-1)$ 
```

When to use:

- Stock returns
 - Growth rates
 - Normalizes values (comparable across scales)
-

Advanced Pandas Concepts

Apply, Map, and Applymap

Confusion matrix:

Function	Works On	Returns	Use Case
<code>apply()</code>	DataFrame or Series	Series or DataFrame	Row/column-wise operations
<code>map()</code>	Series	Series	Element-wise transformations
<code>applymap()</code>	DataFrame	DataFrame	Element-wise (deprecated)

Performance hierarchy (fast to slow):

- 1. **Vectorized operations:** `df['A'] + df['B']` (fastest, always prefer)
- 2. **Built-in functions:** `df.sum()`, `df.mean()`
- 3. **NumPy ufuncs:** `np.sqrt(df)`
- 4. **apply() with NumPy:** `df.apply(np.sum)`
- 5. **apply() with Python:** `df.apply(lambda x: sum(x))`
- 6. **iterrows():** Avoid if possible (slowest)

When to use apply:

- Complex transformations not available as vectorized operations
- Custom functions requiring multiple columns
- Conditional logic too complex for `np.where()`

MultiIndex (Hierarchical Indexing)

Concept: Multiple levels of indices for rows/columns

Example:

		Sales		Profit
Region	Store			
North	A	100	20	
	B	150	30	
South	A	120	25	
	B	180	35	

Why use MultiIndex:

1. **Natural representation:** Some data has inherent hierarchy
2. **Efficient groupby:** Pre-grouped structure
3. **Slicing:** Select data at different levels
4. **Reshaping:** Easy pivot/unstack operations

When to use:

- Panel data (multiple dimensions: time, region, product)
- Results from groupby with multiple keys
- Time series with multiple instruments
- Repeated measures data

Method Chaining

Concept: Consecutive operations on DataFrame

```
python

(df
 .query('sales > 100')
 .groupby('category')
 .agg({'sales': 'sum', 'profit': 'mean'})
 .sort_values('sales', ascending=False)
 .head(10)
 )
```

Advantages:

- Readable pipeline
- No intermediate variables
- Easier to debug (comment out lines)
- Functional programming style

When to use:

- Data cleaning pipelines
 - Exploratory analysis
 - Reproducible workflows
-

Performance Optimization

Why Pandas Can Be Slow

Root causes:

1. **Python loops:** Interpreted, dynamically typed
2. **Object dtype:** Strings stored as Python objects (slow)
3. **Memory allocation:** Growing DataFrames requires reallocation
4. **Type checking:** Runtime type inference overhead

Optimization Strategies

1. Vectorization

Theory: Replace explicit loops with array operations

Mathematical complexity:

Python loop: n operations in Python interpreter = $O(n) \times \text{Python_overhead}$

Vectorized: n operations in C = $O(n) \times C_overhead$

Speed ratio: 10x - 100x faster

Example:

```
python

# Slow - Python loop
result = []
for x in df['A']:
    result.append(x * 2 + 5)

# Fast - Vectorized
result = df['A'] * 2 + 5
```

When vectorization is possible:

- Element-wise operations
- Mathematical transformations
- Boolean logic
- String operations (`.str` accessor)

2. Use Efficient Data Types

Memory and speed impact:

Operation on 1M elements:

int64 (8 bytes):

- Memory: 8 MB
- Speed: baseline

int32 (4 bytes):

- Memory: 4 MB (50% reduction)
- Speed: ~20% faster (better cache utilization)

category (string $\rightarrow$ int mapping):

- Memory: 0.1 MB for 100 unique values (98% reduction)
- Speed: 10x faster for groupby operations

Strategy:

- Use smallest integer type that fits range
- Convert repeated strings to category
- Use float32 for machine learning (acceptable precision)

3. Avoid DataFrame Growth

Why growing is slow:

Append to DataFrame n times:

- Each append creates new memory block
- Copy entire DataFrame each time
- Complexity: $O(n^2)$

Pre-allocate and fill:

- Allocate once
- Fill in place
- Complexity: $O(n)$

Best practice:

python

```
# Bad -  $O(n^2)$ 
df = pd.DataFrame()
for item in items:
    df = df.append(item)

# Good -  $O(n)$ 
results = []
for item in items:
    results.append(item)
df = pd.DataFrame(results)
```

4. Query vs Boolean Indexing

`query()` advantages:

- Optimized for large DataFrames (>15,000 rows)
- Uses numexpr library (faster evaluation)
- More readable for complex conditions

When to use:

```
python

# Small DataFrame - use boolean indexing
df[df['A'] > 5]

# Large DataFrame - use query
df.query('A > 5')

# Complex conditions - query is more readable
df.query('(A > 5) & (B < 10) & (C == "X")')
```

5. Chunking Large Files

Memory limitation problem:

```
File size: 10 GB
Available RAM: 8 GB
Loading entire file: MemoryError
```

Solution - process in chunks:

```
python
```

```
chunk_size = 10000
for chunk in pd.read_csv('large.csv', chunksize=chunk_size):
    # Process each chunk
    result = process(chunk)
    # Save or aggregate
```

When to use:

- File larger than RAM
- Only need aggregated results
- Streaming data processing

6. Use eval for Complex Expressions

Theory: Compile arithmetic expressions for faster evaluation

```
python

# Slower - standard pandas
df['result'] = df['A'] + df['B'] * df['C'] - df['D']

# Faster - compiled expression
df.eval('result = A + B * C - D')
```

Speed improvement:

- 2-3x faster for complex expressions
- Lower memory usage (no intermediate arrays)
- Works best with >10,000 rows

Statistical Concepts in Pandas

Descriptive Statistics

describe() output explained:

```
python

df.describe()
```

Returns these statistics:

- **count:** Number of non-null values (sample size n)

- **mean:** $\mu = (\sum x_i)/n$ - Central tendency
- **std:** $\sigma = \sqrt{(\sum (x_i - \mu)^2)/(n-1))}$ - Standard deviation (spread)
- **min/max:** Range of data
- **25%, 50%, 75%:** Quartiles (Q1, median, Q3)

Interpreting standard deviation:

- $\sigma \approx 0$: Data is clustered around mean
- Large σ : Data is spread out
- For normal distribution: $\sim 68\%$ within $\mu \pm \sigma$, $\sim 95\%$ within $\mu \pm 2\sigma$

Correlation

Mathematical definition:

Pearson correlation coefficient:

$$r = \frac{\sum ((x_i - \bar{x})(y_i - \bar{y}))}{\sqrt{(\sum (x_i - \bar{x})^2 \times \sum (y_i - \bar{y})^2)}}$$

Range: $-1 \leq r \leq 1$

Interpretation:

- $r = 1$: Perfect positive correlation
- $r = -1$: Perfect negative correlation
- $r = 0$: No linear correlation
- $|r| > 0.7$: Strong correlation
- $|r| < 0.3$: Weak correlation

When to use:

python

`df.corr()` # All pairwise correlations

`df['A'].corr(df['B'])` # Specific pair

Use cases:

- Feature selection (remove highly correlated features)
- Identify relationships in data
- Validate hypotheses
- Detect multicollinearity

Caution:

- Correlation $\neq$ Causation
- Only measures linear relationships
- Sensitive to outliers

Covariance

Mathematical definition:

$$\text{Cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)] = \Sigma((x_i - \bar{x})(y_i - \bar{y})) / (n-1)$$

Difference from correlation:

- Covariance: Units depend on variables
- Correlation: Standardized (-1 to 1)
- Relation: $r = \text{Cov}(X, Y) / (\sigma_x \times \sigma_y)$

When to use:

- Portfolio theory (asset covariances)
 - Multivariate statistics
 - PCA preprocessing
-

Data Cleaning Theory

Tidy Data Principles

Hadley Wickham's definition:

1. Each variable forms a column
2. Each observation forms a row
3. Each type of observational unit forms a table

Why it matters:

- Standardized structure $\rightarrow$ easier analysis
- Works seamlessly with groupby, pivot, merge
- Reduces errors in analysis

Example of untidy $\rightarrow$ tidy:

Untidy:

Name	Q1_Sales	Q2_Sales	Q3_Sales
John	100	150	200
Jane	120	140	180

Tidy:

Name	Quarter	Sales
John	Q1	100
John	Q2	150
John	Q3	200
Jane	Q1	120
Jane	Q2	140
Jane	Q3	180

Outlier Detection

Statistical methods:

1. Z-score method:

$$z = (x - \mu) / \sigma$$

Outlier if $|z| > 3$

- Assumes normal distribution
- Threshold typically 2.5 or 3
- Sensitive to extreme outliers (they affect μ and σ)

2. IQR (Interquartile Range) method:

Q1 = 25th percentile

Q3 = 75th percentile

IQR = Q3 - Q1

Outlier if:

$x < Q1 - 1.5 \times IQR$ or $x > Q3 + 1.5 \times IQR$

- Robust to extreme values
- Doesn't assume distribution
- Used in box plots
- Generally preferred method

3. Modified Z-score (MAD - Median Absolute Deviation):

$MAD = \text{median}(|x_i - \text{median}(x)|)$

$\text{Modified z-score} = 0.6745 \times (x_i - \text{median}(x)) / MAD$

Outlier if $|\text{modified z-score}| > 3.5$

- Most robust method
- Resistant to extreme outliers
- Better for skewed distributions

When to remove outliers:

- Data entry errors (impossible values)
- Measurement errors
- Sample from different population

When to keep outliers:

- Genuine extreme values
 - Valuable information (fraud detection)
 - Small sample size (can't afford to lose data)
-

Feature Engineering with Pandas

Creating New Features

1. Mathematical transformations:

```
python

# Log transformation - reduce skewness
df['log_sales'] = np.log(df['sales'] + 1)

# Square root - moderate skewness
df['sqrt_age'] = np.sqrt(df['age'])

# Polynomial features
df['sales_squared'] = df['sales'] ** 2
```

Why transform:

- Linearize relationships
- Normalize distributions

- Reduce impact of outliers
- Meet model assumptions

2. Binning (Discretization):

```
python

# Equal-width bins
pd.cut(df['age'], bins=5)

# Equal-frequency bins (quantiles)
pd.qcut(df['income'], q=4)

# Custom bins
pd.cut(df['age'], bins=[0, 18, 65, 100],
      labels=['minor', 'adult', 'senior'])
```

When to bin:

- Capture non-linear relationships
- Reduce noise in continuous variables
- Create interpretable categories
- Handle outliers

3. Interaction features:

```
python

df['price_per_sqft'] = df['price'] / df['sqft']
df['total_rooms'] = df['bedrooms'] + df['bathrooms']
df['interaction'] = df['A'] * df['B']
```

Purpose:

- Capture combined effects
- Improve model performance
- Domain knowledge incorporation

Encoding Categorical Variables

1. Label Encoding:

```
python

df['category_encoded'] = df['category'].astype('category').cat.codes
```

When to use:

- Ordinal variables (small < medium < large)
- Tree-based models (decision trees, random forests)

Problems:

- Implies ordering (A=0, B=1, C=2 suggests $C > B > A$)
- Not suitable for non-ordinal categories

2. One-Hot Encoding:

```
python  
  
pd.get_dummies(df['category'])
```

Mathematical representation:

Original:	One-Hot Encoded:		
category	cat_A	cat_B	cat_C
A	1	0	0
B	0	1	0
C	0	0	1

When to use:

- Nominal variables (no order)
- Linear models, neural networks
- Few unique categories (<10)

Problems:

- Curse of dimensionality (many categories $\rightarrow$ many columns)
- Dummy variable trap (perfect multicollinearity)
- Solution: Drop one category (`drop_first=True`)

3. Frequency Encoding:

```
python  
  
freq = df['category'].value_counts()  
df['category_freq'] = df['category'].map(freq)
```

When to use:

- Many categories
- Frequency is informative
- Reduce dimensionality

4. Target Encoding:

```
python

mean_target = df.groupby('category')['target'].mean()
dff['category_encoded'] = dff['category'].map(mean_target)
```

When to use:

- Categorical with many levels
 - Strong relationship with target
 - Careful: Risk of overfitting (use cross-validation)
-

Window Functions Theory

Cumulative Operations

Cumulative sum:

Mathematical: $S(n) = \sum_{i=1}^n x_i$

Data: [1, 2, 3, 4, 5]

Cumsum: [1, 3, 6, 10, 15]

When to use:

- Running totals (accumulated sales)
- Cumulative statistics
- Ranking over time

Cumulative product:

$P(n) = \prod_{i=1}^n x_i$

Example - compound returns:

Returns: [1.05, 1.03, 1.02]

Cumulative: [1.05, 1.0815, 1.10313]

Use cases:

- Compound interest
- Growth factors
- Probability chains

Ranking

Ranking methods:

```
python  
  
df['rank'] = df['score'].rank()
```

Methods (for ties):

1. **average:** $(1+2)/2 = 1.5$ for both
2. **min:** Both get 1
3. **max:** Both get 2
4. **first:** Order of appearance
5. **dense:** 1, 1, 2 (no gaps)

When to use:

- Percentile calculation
- Competition results
- Creating ordinal features

Expanding Windows

Concept: Window starts at first row and grows

```
python  
  
df['expanding_mean'] = df['value'].expanding().mean()
```

Mathematical:

For time t:

Expanding mean = $(1/t) \times \sum_{i=1}^t x_i$

t=1: x_1

t=2: $(x_1 + x_2) / 2$

t=3: $(x_1 + x_2 + x_3) / 3$

When to use:

- All-time statistics
- Historical averages
- Baseline comparison

Difference from rolling:

- Rolling: Fixed window size
 - Expanding: Growing window size
-

Query Optimization

Index Theory

What is an index:

- Data structure (usually B-tree or hash table)
- Maps values to row locations
- Trades space for speed

Complexity:

Without index:

- Search: $O(n)$ - scan entire DataFrame
- Filter: $O(n)$

With index:

- Search: $O(\log n)$ - binary search
- Filter: $O(\log n + k)$ where k = result size

When to index:

- Frequent filtering on column
- Merging/joining on column

- Groupby on column
- Time series (datetime index)

Cost:

- Additional memory (~20% overhead)
- Slower inserts/updates
- Only helps for indexed columns

Best practices:

```
python

# Index before repeated operations
df = df.set_index('id')
result1 = df.loc[123] # Fast
result2 = df.loc[456] # Fast

# Reset index when no longer needed
df = df.reset_index()
```

Filter Order Matters

Optimization principle: Apply most selective filters first

```
python

# Less efficient
df[(df['B'] < 100) & (df['A'] == 'rare_value')]

# More efficient (if 'A' is very selective)
df[(df['A'] == 'rare_value') & (df['B'] < 100)]
```

Why:

- Short-circuit evaluation
- Smaller intermediate results
- Better memory usage

Rule of thumb:

1. Equality filters first (most selective)
2. Range filters second
3. Expensive operations last (string matching, regex)

Memory Management

View vs Copy

Critical concept: Some operations return views (reference), others return copies

View (shares memory):

```
python

subset = df[['A', 'B']] # Copy
row_slice = df[10:20]   # View (usually)
column = df['A']        # View
```

Why it matters:

```
python

# Modifying view affects original
row_slice = df[10:20]
row_slice['A'] = 99 # Changes df!

# Solution: Explicit copy
row_slice = df[10:20].copy()
row_slice['A'] = 99 # df unchanged
```

Checking:

```
python

subset._is_view # Check if view
```

Best practice:

- Use `.copy()` when creating subset for modification
- Understand view vs copy behavior
- Watch for `SettingWithCopyWarning`

Memory Profiling

Check memory usage:

```
python
```



```
df.info(memory_usage='deep') # Actual memory usage
df.memory_usage(deep=True).sum() # Total bytes
```

Optimization workflow:

```
python

# 1. Check current usage
initial = df.memory_usage(deep=True).sum()

# 2. Optimize types
df['int_col'] = df['int_col'].astype('int32')
df['cat_col'] = df['cat_col'].astype('category')

# 3. Check improvement
final = df.memory_usage(deep=True).sum()
reduction = (initial - final) / initial * 100
print(f'Reduced by {reduction:.1f}%')
```

Best Practices Summary

Code Organization

1. Readability:

```
python

# Bad
x=df[df.A>5&df.B<10].groupby('C').D.sum()

# Good
filtered = df[(df['A'] > 5) & (df['B'] < 10)]
result = filtered.groupby('C')['D'].sum()

# Best - method chaining
result = (df
    .query('A > 5 and B < 10')
    .groupby('C')['D']
    .sum()
)
```

2. Avoid magic numbers:

```
python
```

```
# Bad
```

```
df = df[df['age'] < 120]
```

```
# Good
```

```
MAX_REASONABLE_AGE = 120
```

```
df = df[df['age'] < MAX_REASONABLE_AGE]
```

3. Validate assumptions:

```
python
```

```
assert df['price'].min() > 0, "Negative prices found"
```

```
assert df['date'].is_monotonic_increasing, "Dates not sorted"
```

```
assert df.index.is_unique, "Duplicate indices"
```

Performance Checklist

1. **Use vectorized operations** instead of loops
2. **Choose appropriate data types** (int32, category)
3. **Index columns** used for frequent filtering/joining
4. **Pre-allocate DataFrames** instead of growing
5. **Use query()** for complex filters on large data
6. **Process in chunks** for files larger than RAM
7. **Profile before optimizing** - measure, don't guess

Data Quality Checks

Standard workflow:

```
python
```

1. Shape and types

```
df.shape  
df.dtypes  
df.info()
```

2. Missing data

```
df.isnull().sum()  
df.isnull().mean() # Percentage missing
```

3. Duplicates

```
df.duplicated().sum()
```

4. Value ranges

```
df.describe()  
df['numeric_col'].quantile([0.01, 0.99]) # Check extremes
```

5. Cardinality

```
df.nunique()
```

6. Sample inspection

```
df.sample(10)  
df.head()
```

When to Use What: Decision Trees

Choosing Between NumPy and Pandas

Start

```
|  
├─ Homogeneous numeric data? → NumPy  
|   └─ Need labels/mixed types? → Convert to Pandas  
|  
├─ Tabular data with labels? → Pandas  
|   └─ Need maximum speed? → Extract .values to NumPy  
|  
└─ Time series? → Pandas (datetime index)
```

Choosing Aggregation Method

Aggregation Task

```
|  
├─ Summary statistics? → describe()  
└─ Group-wise? → groupby()
```

- └─ Time-based? → `resample()`
- └─ Cross-tabulation? → `pivot_table()` or `crosstab()`
- └─ Rolling statistics? → `rolling()`

Choosing Join Method

Merging DataFrames

- |
- └─ Need all from both? → `how='outer'`
- └─ Keep all from left? → `how='left'`
- └─ Only matching? → `how='inner'`
- └─ Simple concat? → `pd.concat()`
- └─ Join on index? → `df1.join(df2)`

This guide provides the theoretical foundation for understanding when, why, and how to use NumPy and Pandas operations effectively in real-world data analysis scenarios.